In [4]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [5]:
```
df = pd.read_csv(r"BAJFINANCE.csv")
df.head()
```

Out[5]:

	Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	Volume	Turnover	Trades	Deliver Vo
0	2000-01-03	BAJAUTOFIN	EQ	46.95	49.45	50.75	46.5	50.75	50.75	50.05	7600	3.803800e+10	NaN	
1	2000-01-04	BAJAUTOFIN	EQ	50.75	53.20	53.20	47.9	48.00	48.10	48.56	5000	2.428000e+10	NaN	
2	2000-01-05	BAJAUTOFIN	EQ	48.10	46.55	47.40	44.6	44.60	44.60	45.47	3500	1.591450e+10	NaN	
3	2000-01-06	BAJAUTOFIN	EQ	44.60	43.50	46.00	42.1	46.00	45.25	44.43	6200	2.754750e+10	NaN	
4	2000-01-07	BAJAUTOFIN	EQ	45.25	48.00	48.00	42.0	42.90	42.90	44.44	3500	1.555550e+10	NaN	

In [6]:
```
df.set_index('Date')
```

Out[6]:

Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	Volume	Turnover	T
2000-01-03	BAJAUTOFIN	EQ	46.95	49.45	50.75	46.50	50.75	50.75	50.05	7600	3.803800e+10	
2000-01-04	BAJAUTOFIN	EQ	50.75	53.20	53.20	47.90	48.00	48.10	48.56	5000	2.428000e+10	
2000-01-05	BAJAUTOFIN	EQ	48.10	46.55	47.40	44.60	44.60	44.60	45.47	3500	1.591450e+10	
2000-01-06	BAJAUTOFIN	EQ	44.60	43.50	46.00	42.10	46.00	45.25	44.43	6200	2.754750e+10	
2000-01-07	BAJAUTOFIN	EQ	45.25	48.00	48.00	42.00	42.90	42.90	44.44	3500	1.555550e+10	
...	...	...	...	...	...	...	...	...	...	...	...	
2020-08-25	BAJFINANCE	EQ	3492.05	3525.00	3660.00	3510.00	3658.00	3642.90	3579.12	9854070	3.526895e+15	33
2020-08-26	BAJFINANCE	EQ	3642.90	3665.00	3707.00	3631.00	3638.20	3645.55	3668.17	6665336	2.444958e+15	21
2020-08-27	BAJFINANCE	EQ	3645.55	3656.95	3668.40	3596.40	3636.00	3632.50	3631.13	4611132	1.674361e+15	16
2020-08-28	BAJFINANCE	EQ	3632.50	3650.00	3688.00	3617.05	3672.05	3670.80	3652.77	4251575	1.553003e+15	13
2020-08-31	BAJFINANCE	EQ	3670.80	3715.00	3749.85	3465.00	3478.50	3487.80	3602.93	8529788	3.073224e+15	29

5070 rows × 14 columns

In [7]:
```
df.describe()
```

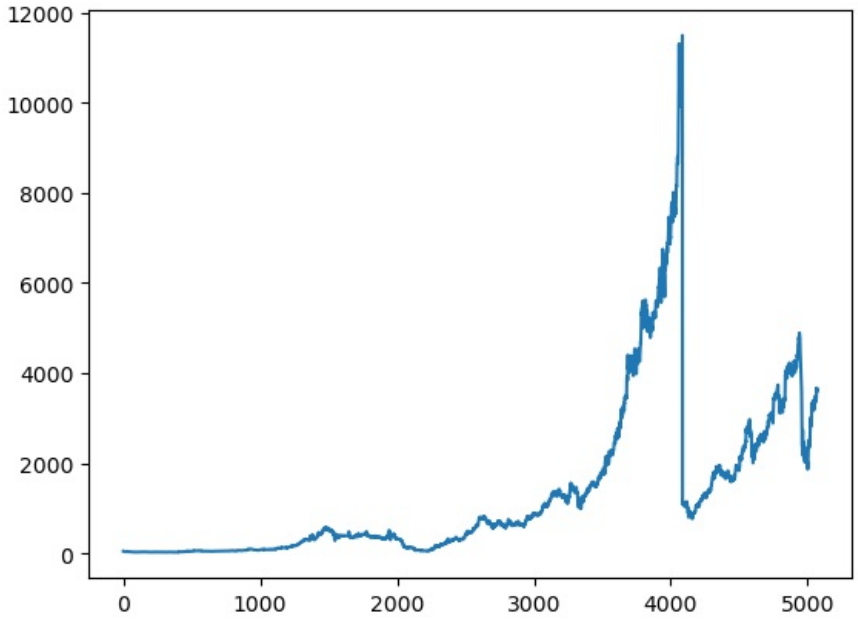
Out[7]:

	Prev Close	Open	High	Low	Last	Close	VWAP	Vo
count	5070.000000	5070.000000	5070.000000	5070.000000	5070.000000	5070.000000	5070.000000	5.070000
mean	1311.476312	1312.330276	1334.300828	1289.349586	1312.278836	1312.154980	1312.440830	5.060571
std	1782.013137	1781.955970	1808.504414	1753.704145	1782.393446	1782.186659	1781.824954	1.798886
min	24.500000	25.200000	25.200000	24.500000	24.500000	24.500000	25.200000	3.000000
25%	105.400000	104.275000	108.625000	100.725000	106.937500	107.725000	106.685000	4.697750
50%	536.375000	534.975000	552.000000	522.150000	536.400000	537.725000	537.990000	1.567200
75%	1757.525000	1764.012500	1784.600000	1740.550000	1760.037500	1757.750000	1758.297500	8.194950
max	11393.300000	11300.000000	11770.000000	11294.000000	11386.700000	11393.300000	11490.730000	2.596010

Plotting the target variable VWAP over time

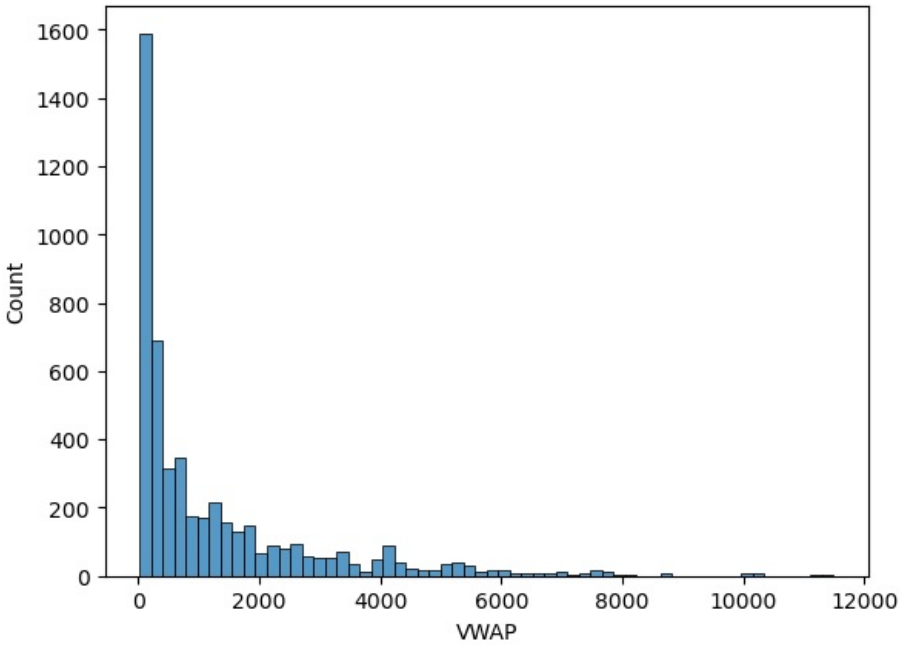
In [8]:
```
df['VWAP'].plot()
```

Out[8]: <Axes: >



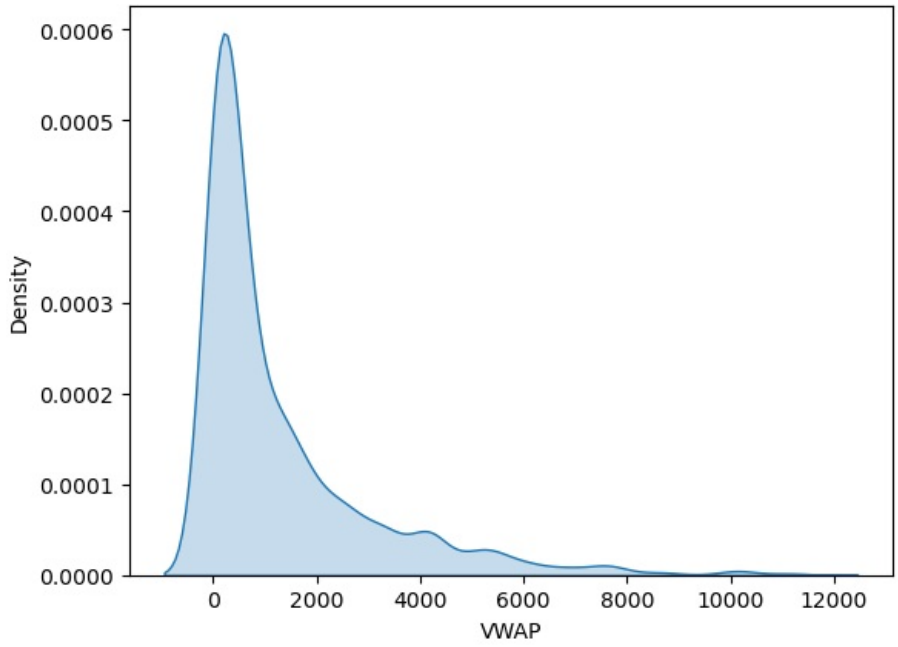
In [9]:
```
sns.histplot(data=df, x ='VWAP')
```

Out[9]: <Axes: xlabel='VWAP', ylabel='Count'>



In [10]:
```
sns.kdeplot(data=df, x='VWAP', fill='True')
```

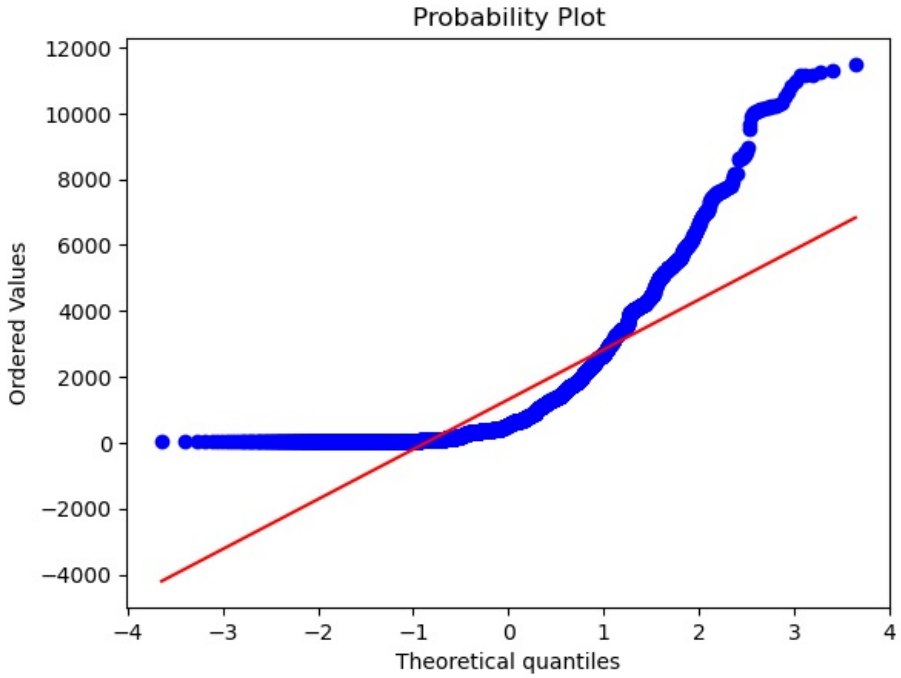
Out[10]: <Axes: xlabel='VWAP', ylabel='Density'>



In [11]:
```
from scipy import stats
```

In [12]:
```
stats.probplot(x = df["VWAP"], plot = plt)
```

Out[12]:
```
((array([-3.63926987, -3.40416755, -3.27460203, ...,  3.27460203,
          3.40416755,  3.63926987]),
  array([   25.2 ,    25.25,    25.35, ..., 11243.26, 11304.33, 11490.73])),
 (np.float64(1516.21990136437),
  np.float64(1312.4408303747532),
  np.float64(0.8504596915638712)))
```



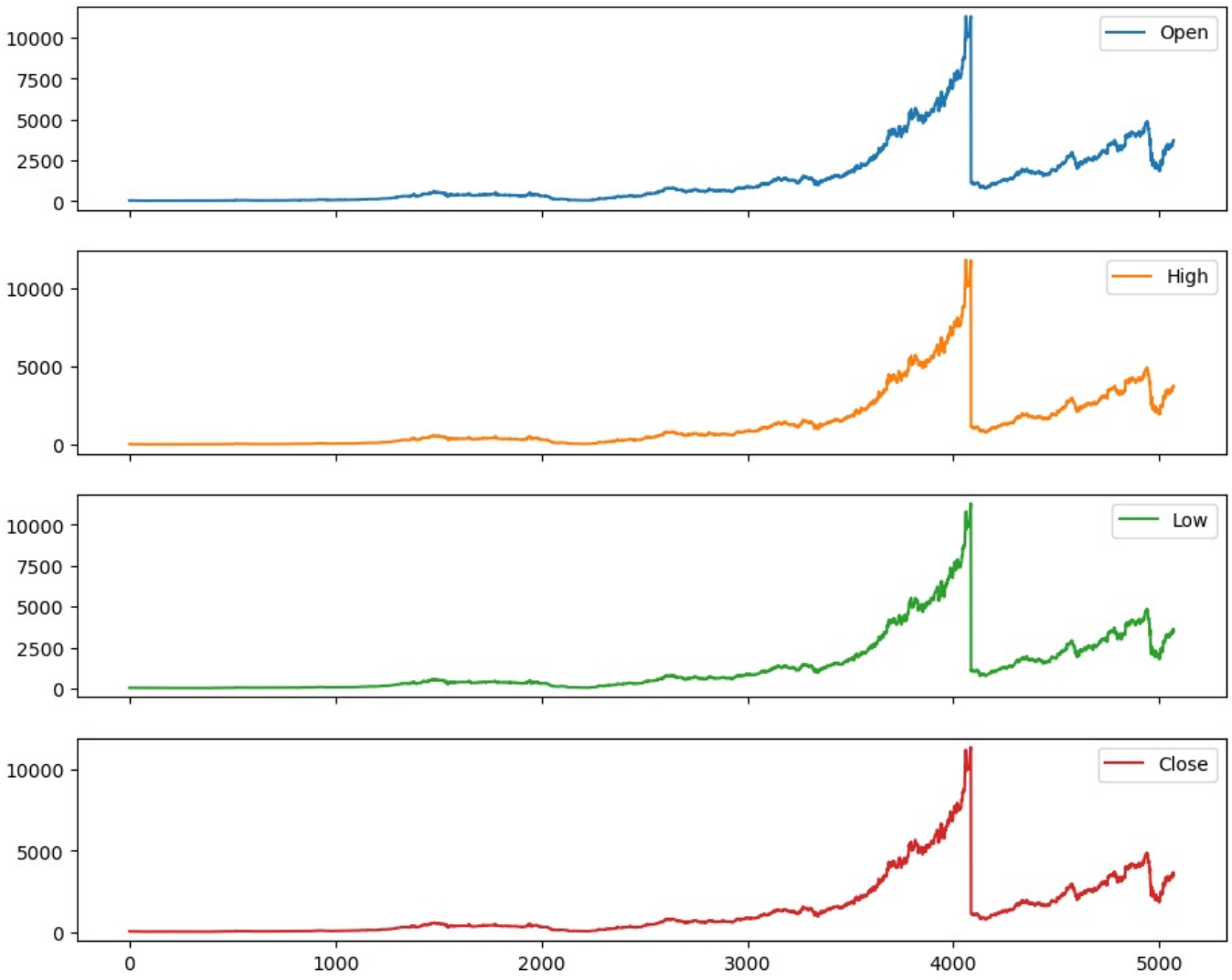
In [13]:
```
df.columns
```

Out[13]:
```
Index(['Date', 'Symbol', 'Series', 'Prev Close', 'Open', 'High', 'Low', 'Last',
       'Close', 'VWAP', 'Volume', 'Turnover', 'Trades', 'Deliverable Volume',
       '%Deliverble'],
      dtype='object')
```

In [14]:
```
cols=['Open','High','Low','Close']
```

In [15]:
```
df[cols].plot(figsize=(11,9),subplots= True)
```

Out[15]:
```
array([<Axes: >, <Axes: >, <Axes: >, <Axes: >], dtype=object)
```



In [16]:
```
#Candle Stick Graph

import plotly.graph_objects as go

fig = go.Figure(
    data=[
        go.Candlestick(
            x=df.index[:50],
            open=df['Open'][:50],
            high=df['High'][:50],
            low=df['Low'][:50],
            close=df['Close'][:50]
        )
    ]
)

fig.show()
```

In [17]:
```
fig.update_layout(xaxis_rangeslider_visible = False )
```

so u can observe here some kind of Seasonality

Feature Engineering Almost every time series problem will have some external features or some internal feature engineering to help the model. Let's add some basic features like lag values of available numeric features that are widely used for time series problems. Since we need to predict the price of the stock for a day, we cannot use the feature values of the same day since they will be unavailable at actual inference time. We need to use statistics like mean, standard deviation of their lagged values. We will use three sets of lagged values, one previous day, one looking back 7 days and another looking back 30 days as a proxy for last week and last month metrics.

Data Pre-Processing

In [18]:
```
df.shape
```

Out[18]:
```
(5070, 15)
```

In [19]:
```
df.isna().sum()
```

Out[19]:
```
Date                    0
Symbol                  0
Series                  0
Prev Close              0
Open                    0
High                    0
Low                     0
Last                    0
Close                   0
VWAP                    0
Volume                  0
Turnover                0
Trades                2779
Deliverable Volume     446
%Deliverble            446
dtype: int64
```

In [20]:
```
df.dropna(inplace=True)
```

In [21]:
```
df.isna().sum()
```

Out[21]:
```
Date                  0
Symbol                0
Series                0
Prev Close            0
Open                  0
High                  0
Low                   0
Last                  0
Close                 0
VWAP                  0
Volume                0
Turnover              0
Trades                0
Deliverable Volume    0
%Deliverble           0
dtype: int64
```

In [22]:
```
df.shape
```

Out[22]:
```
(2291, 15)
```

In [23]:
```
data=df.copy()
```

In [24]:
```
data.dtypes
```

Out[24]:
```
Date                   object
Symbol                 object
Series                 object
Prev Close            float64
Open                  float64
High                  float64
Low                   float64
Last                  float64
Close                 float64
VWAP                  float64
Volume                  int64
Turnover              float64
Trades                float64
Deliverable Volume    float64
%Deliverble           float64
dtype: object
```

In [25]:
```
data.columns
```

Out[25]:
```
Index(['Date', 'Symbol', 'Series', 'Prev Close', 'Open', 'High', 'Low', 'Last',
       'Close', 'VWAP', 'Volume', 'Turnover', 'Trades', 'Deliverable Volume',
       '%Deliverble'],
      dtype='object')
```

In [26]:
```
data[['Open', 'High', 'Low', 'Close']].corr()
```

Out[26]:

	Open	High	Low	Close
Open	1.000000	0.999575	0.999602	0.999285
High	0.999575	1.000000	0.999358	0.999695
Low	0.999602	0.999358	1.000000	0.999616
Close	0.999285	0.999695	0.999616	1.000000

In [ ]:

In [27]:
```
lag_features=['High','Low','Volume','Turnover','Trades']
window1=3
window2=7
```

In [28]:
```
data["High"].rolling(window = 3).mean()
```

Out[28]:
```
2779           NaN
2780           NaN
2781     637.733333
2782     639.233333
2783     634.250000
            ...
5065    3538.333333
5066    3627.333333
5067    3678.466667
5068    3687.800000
5069    3702.083333
Name: High, Length: 2291, dtype: float64
```

In [29]:
```
for feature in lag_features:
    data[feature+'rolling_mean_3']=data[feature].rolling(window=window1).mean()
    data[feature+'rolling_mean_7']=data[feature].rolling(window=window2).mean()
```

In [30]:
```
data.columns
```

Out[30]:
```
Index(['Date', 'Symbol', 'Series', 'Prev Close', 'Open', 'High', 'Low', 'Last',
       'Close', 'VWAP', 'Volume', 'Turnover', 'Trades', 'Deliverable Volume',
       '%Deliverble', 'Highrolling_mean_3', 'Highrolling_mean_7',
       'Lowrolling_mean_3', 'Lowrolling_mean_7', 'Volumerolling_mean_3',
       'Volumerolling_mean_7', 'Turnoverrolling_mean_3',
       'Turnoverrolling_mean_7', 'Tradesrolling_mean_3',
       'Tradesrolling_mean_7'],
      dtype='object')
```

In [31]:
```
data.head(4)
```

Out[31]:

	Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	...	Highrolling_mean_3	High
2779	2011-06-01	BAJFINANCE	EQ	616.70	617.00	636.5	616.00	627.00	631.85	627.01	...	NaN	
2780	2011-06-02	BAJFINANCE	EQ	631.85	625.00	638.9	620.00	634.00	633.45	636.04	...	NaN	
2781	2011-06-03	BAJFINANCE	EQ	633.45	625.15	637.8	620.00	623.00	625.00	625.09	...	637.733333	
2782	2011-06-06	BAJFINANCE	EQ	625.00	620.00	641.0	611.35	611.35	614.00	616.03	...	639.233333	

4 rows × 25 columns

In [32]:
```
for feature in lag_features:
    data[feature+'rolling_std_3']=data[feature].rolling(window=window1).std()
    data[feature+'rolling_std_7']=data[feature].rolling(window=window2).std()
```

In [33]:
```
data.head()
```

Out[33]:

	Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	...	Highrolling_std_3	Highr
2779	2011-06-01	BAJFINANCE	EQ	616.70	617.00	636.50	616.00	627.00	631.85	627.01	...	NaN	
2780	2011-06-02	BAJFINANCE	EQ	631.85	625.00	638.90	620.00	634.00	633.45	636.04	...	NaN	
2781	2011-06-03	BAJFINANCE	EQ	633.45	625.15	637.80	620.00	623.00	625.00	625.09	...	1.201388	
2782	2011-06-06	BAJFINANCE	EQ	625.00	620.00	641.00	611.35	611.35	614.00	616.03	...	1.625833	
2783	2011-06-07	BAJFINANCE	EQ	614.00	604.00	623.95	604.00	619.90	619.15	617.73	...	9.062422	

5 rows × 35 columns

In [34]:
```
data.columns
```

Out[34]:
```
Index(['Date', 'Symbol', 'Series', 'Prev Close', 'Open', 'High', 'Low', 'Last',
       'Close', 'VWAP', 'Volume', 'Turnover', 'Trades', 'Deliverable Volume',
       '%Deliverble', 'Highrolling_mean_3', 'Highrolling_mean_7',
       'Lowrolling_mean_3', 'Lowrolling_mean_7', 'Volumerolling_mean_3',
       'Volumerolling_mean_7', 'Turnoverrolling_mean_3',
       'Turnoverrolling_mean_7', 'Tradesrolling_mean_3',
       'Tradesrolling_mean_7', 'Highrolling_std_3', 'Highrolling_std_7',
       'Lowrolling_std_3', 'Lowrolling_std_7', 'Volumerolling_std_3',
       'Volumerolling_std_7', 'Turnoverrolling_std_3', 'Turnoverrolling_std_7',
       'Tradesrolling_std_3', 'Tradesrolling_std_7'],
      dtype='object')
```

In [35]:
```
data.shape
```

Out[35]:
```
(2291, 35)
```

In [36]:
```
data.isna().sum()
```

Out[36]:
```
Date                     0
Symbol                   0
Series                   0
Prev Close               0
Open                     0
High                     0
Low                      0
Last                     0
Close                    0
VWAP                     0
Volume                   0
Turnover                 0
Trades                   0
Deliverable Volume       0
%Deliverble              0
Highrolling_mean_3       2
Highrolling_mean_7       6
Lowrolling_mean_3        2
Lowrolling_mean_7        6
Volumerolling_mean_3     2
Volumerolling_mean_7     6
Turnoverrolling_mean_3   2
Turnoverrolling_mean_7   6
Tradesrolling_mean_3     2
Tradesrolling_mean_7     6
Highrolling_std_3        2
Highrolling_std_7        6
Lowrolling_std_3         2
Lowrolling_std_7         6
Volumerolling_std_3      2
Volumerolling_std_7      6
Turnoverrolling_std_3    2
Turnoverrolling_std_7    6
Tradesrolling_std_3      2
Tradesrolling_std_7      6
dtype: int64
```

In [37]:
```
data.dropna(inplace=True)
```

In [38]:
```
data.columns
```

Out[38]:
```
Index(['Date', 'Symbol', 'Series', 'Prev Close', 'Open', 'High', 'Low', 'Last',
       'Close', 'VWAP', 'Volume', 'Turnover', 'Trades', 'Deliverable Volume',
       '%Deliverble', 'Highrolling_mean_3', 'Highrolling_mean_7',
       'Lowrolling_mean_3', 'Lowrolling_mean_7', 'Volumerolling_mean_3',
       'Volumerolling_mean_7', 'Turnoverrolling_mean_3',
       'Turnoverrolling_mean_7', 'Tradesrolling_mean_3',
       'Tradesrolling_mean_7', 'Highrolling_std_3', 'Highrolling_std_7',
       'Lowrolling_std_3', 'Lowrolling_std_7', 'Volumerolling_std_3',
       'Volumerolling_std_7', 'Turnoverrolling_std_3', 'Turnoverrolling_std_7',
       'Tradesrolling_std_3', 'Tradesrolling_std_7'],
      dtype='object')
```

In [39]:
```
ind_features=['Highrolling_mean_3', 'Highrolling_mean_7',
       'Lowrolling_mean_3', 'Lowrolling_mean_7', 'Volumerolling_mean_3',
       'Volumerolling_mean_7', 'Turnoverrolling_mean_3',
       'Turnoverrolling_mean_7', 'Tradesrolling_mean_3',
       'Tradesrolling_mean_7', 'Highrolling_std_3', 'Highrolling_std_7',
       'Lowrolling_std_3', 'Lowrolling_std_7', 'Volumerolling_std_3',
       'Volumerolling_std_7', 'Turnoverrolling_std_3', 'Turnoverrolling_std_7',
       'Tradesrolling_std_3', 'Tradesrolling_std_7']
```

In [40]:
```
data.shape
```

Out[40]: (2285, 35)

In [41]:
```
training_data=data[0:1800]
test_data=data[1800:]
```

In [42]:
```
training_data
```

Out[42]:

	Date	Symbol	Series	Prev Close	Open	High	Low	Last	Close	VWAP	...	Highrolling_std_3
2785	2011-06-09	BAJFINANCE	EQ	635.60	639.80	647.00	630.00	630.00	631.10	638.27	...	12.769789
2786	2011-06-10	BAJFINANCE	EQ	631.10	641.85	648.25	618.55	621.10	622.20	634.16	...	1.639360
2787	2011-06-13	BAJFINANCE	EQ	622.20	616.00	627.85	616.00	622.75	624.95	622.92	...	11.434196
2788	2011-06-14	BAJFINANCE	EQ	624.95	625.00	628.95	619.95	621.20	622.10	625.35	...	11.473593
2789	2011-06-15	BAJFINANCE	EQ	622.10	612.00	623.00	598.10	605.00	601.70	606.90	...	3.165833
...	...	...	...	...	...	...	...	...	...	...	...	...
4580	2018-09-04	BAJFINANCE	EQ	2724.05	2724.00	2777.65	2683.50	2748.00	2746.30	2726.23	...	88.954937
4581	2018-09-05	BAJFINANCE	EQ	2746.30	2740.15	2764.80	2668.00	2704.45	2716.90	2712.53	...	63.129081
4582	2018-09-06	BAJFINANCE	EQ	2716.90	2729.00	2731.50	2671.40	2672.20	2684.10	2695.89	...	23.818183
4583	2018-09-07	BAJFINANCE	EQ	2684.10	2698.40	2751.40	2672.60	2745.00	2744.20	2716.32	...	16.755397
4584	2018-09-10	BAJFINANCE	EQ	2744.20	2732.00	2738.00	2596.00	2607.60	2615.65	2655.39	...	10.147413

1800 rows × 35 columns

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [43]:
```
## Install pmdarima
```

In [44]:
```
##!pip install pmdarima

## if pip install pmdarima fails to you, try as a user ( refer any one of the below commands in Jupyter Noteboo
## pip install pmdarima --user
## !pip install pmdarima==1.7.1
## !pip install pmdarima==1.7.1 --user


## U can also refer Anaconda Prompt to download pmdarima , refer below command in Anaonda Prompt :
## conda install -c saravji pmdarima
## (where c is name of my channel)
```

In [ ]:

In [ ]:

In [ ]:

In [45]:
```
from pmdarima import auto_arima
```

In [46]:
```
import pmdarima
pmdarima.__version__
```

```
## 1.7.1
## U can encounter different pmdarima version !
```

Out[46]: '2.1.1'

In [ ]:

In [47]:
```
import warnings
warnings.filterwarnings('ignore')
```

In [48]:
```
model = auto_arima(y=training_data['VWAP'], X=training_data[ind_features], trace=True)
```

```
Performing stepwise search to minimize aic
 ARIMA(2,0,2)(0,0,0)[0] intercept   : AIC=20931.542, Time=4.60 sec
 ARIMA(0,0,0)(0,0,0)[0] intercept   : AIC=20925.228, Time=2.37 sec
 ARIMA(1,0,0)(0,0,0)[0] intercept   : AIC=20926.352, Time=2.62 sec
 ARIMA(0,0,1)(0,0,0)[0] intercept   : AIC=20926.324, Time=3.74 sec
 ARIMA(0,0,0)(0,0,0)[0]             : AIC=32616.913, Time=2.16 sec
 ARIMA(1,0,1)(0,0,0)[0] intercept   : AIC=20929.238, Time=3.84 sec

Best model:  ARIMA(0,0,0)(0,0,0)[0] intercept
Total fit time: 19.380 seconds
```

In [50]:
```
model.fit(y=training_data['VWAP'], X=training_data[ind_features])

##  model.fit(y=training_data['VWAP'] , exogenous= training_data[ind_features])
```

Out[50]:

ARIMA
ARIMA(0,0,0)(0,0,0)[0] intercept

In [ ]:

In [ ]:

In [ ]:

In [51]:
```
# doing Forecast

forecast = model.predict(n_periods=len(test_data), X=test_data[ind_features])

## forecast = model.predict(n_periods=len(test_data), exogenous=test_data[ind_features])
```

In [52]:
```
forecast
```

Out[52]:
```
1800    2600.732684
1801    2625.147876
1802    2600.996129
1803    2556.412399
1804    2572.872101
           ...
2280    3447.680799
2281    3677.852747
2282    3685.097352
2283    3583.718176
2284    3392.936430
Length: 485, dtype: float64
```

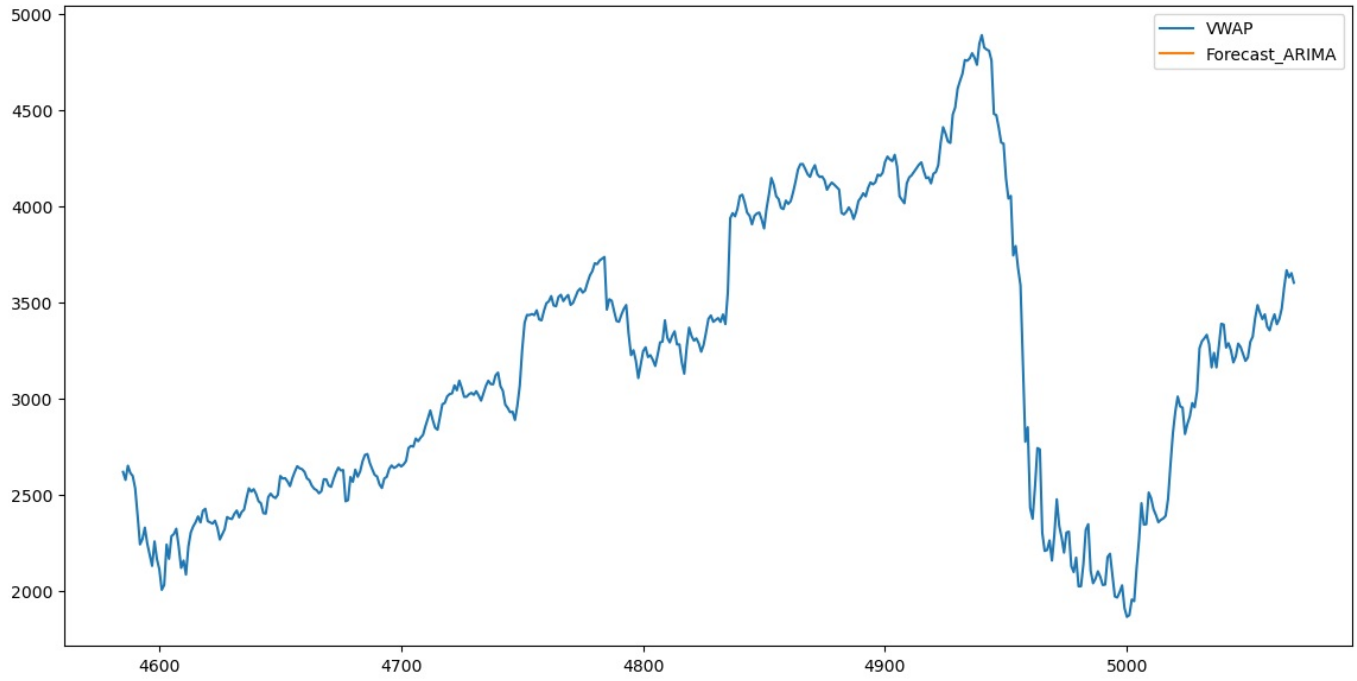
In [53]:
```
test_data['Forecast_ARIMA']= forecast
```

In [54]:
```
test_data['Forecast_ARIMA']
```

Out[54]:
```
4585   NaN
4586   NaN
4587   NaN
4588   NaN
4589   NaN
        ..
5065   NaN
5066   NaN
5067   NaN
5068   NaN
5069   NaN
Name: Forecast_ARIMA, Length: 485, dtype: float64
```

In [55]:
```
test_data[['VWAP','Forecast_ARIMA']].plot(figsize=(14,7))
```

Out[55]: <Axes: >



In [ ]:

In [ ]:

The Auto ARIMA model seems to do a fairly good job in predicting the stock price

In [ ]:

Checking Accuracy of our model

In [56]:
```
from sklearn.metrics import mean_absolute_error, mean_squared_error
```

In [ ]:
```
np.sqrt(mean_squared_error(test_data['VWAP'],test_data['Forecast_ARIMA']))
```

In [ ]:
```
mean_absolute_error(test_data['VWAP'],test_data['Forecast_ARIMA'])
```

In [66]:
```
### mean, var, std--> gives worst prediction RMSE 1484, MAE 1228
### mean --> gives 122 RMSE, gives 82 MAE
### mean,std--> gives 120,79
```